

Caching

For databases that store data on disk, we can cache frequent or recent queries in memory. A **cache** stores temporary data in fast storage (RAM) so repeated requests are served quickly instead of querying the database every time.

1. What is Caching?

Caching means storing frequently accessed data in memory for faster retrieval.

Instead of:

None

User request

→ Database query

→ Return result

Use:

None

User request

→ Cache hit

→ Return result instantly

2. Why It Matters

Databases use disk storage which is slower than memory.

Each DB query may involve:

None

Disk I/O

Network call

Locks

CPU processing

Indexes scan

Replication overhead

Cache avoids repeated cost.

3. Main Benefits of Caching

A. Performance

Memory is much faster than disk.

None

Cache latency = milliseconds / microseconds

DB latency = higher

B. Availability

If DB is temporarily slow/down:

None

Serve cached data

(Only if requested data exists in cache)

C. Scalability

Cache absorbs repeated traffic.

None

10,000 requests

Only 500 reach DB

Rest served by cache

Reduces backend load.

4. Basic Flow

None

Client Request



Check Cache



Hit → Return Data

Miss → Query DB → Store in Cache → Return

5. Cache Hit vs Cache Miss

Cache Hit

None

Requested data found in cache

Fast response.

Cache Miss

None

Data not found

Need DB query

Slower first request.

6. Example

Without cache:

None

Product page opened 10,000 times

10,000 DB queries

With cache:

None

1 DB query

9,999 cache hits

7. Where Caching Can Be Done

A. Client-side Cache

Browser stores:

None

Images

CSS

JS

API responses

B. API Gateway Cache

None

Client

↓

API Gateway Cache

↓

Services

Shared cache layer before services.

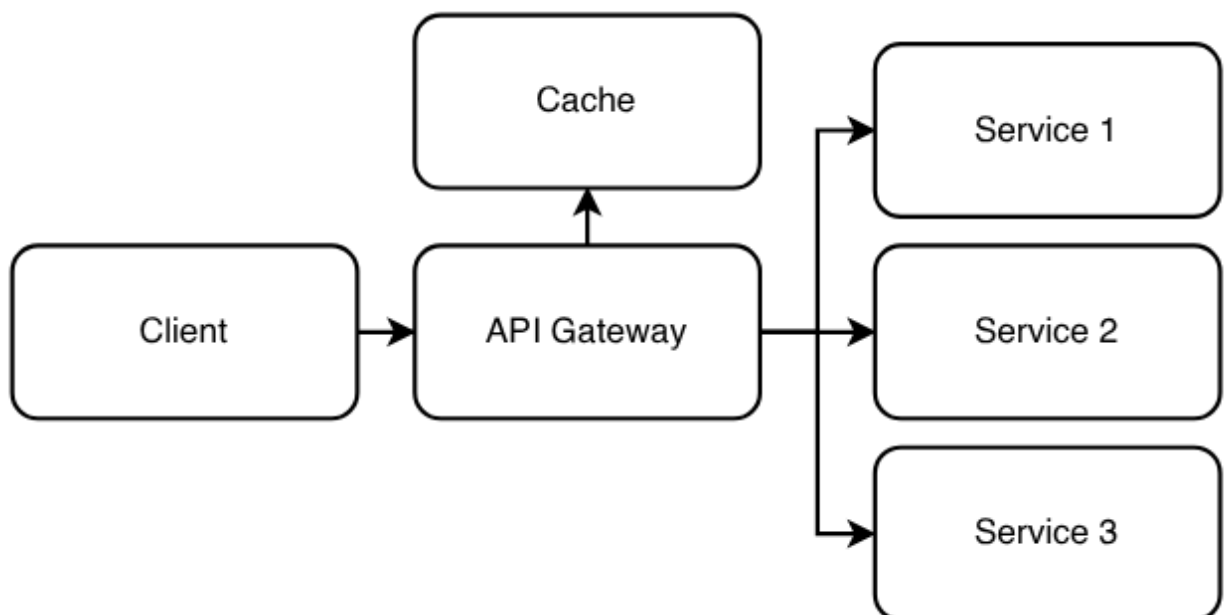
C. Service-Level Cache

Each microservice keeps local/shared cache.

D. Database Query Cache

Inside DB engine.

8. API Gateway Caching Architecture



None

Client



API Gateway



Cache



Service 1

Service 2

Service 3

Benefits:

None

Independent scaling

Reduced backend traffic

Lower latency

9. Popular Cache Technologies

- Redis
- Memcached
- Hazelcast
- CDN caches
- Browser cache

10. Cache Strategies

A. Cache Aside (Lazy Loading)

None

App checks cache first

Miss → DB → update cache

Most common.

B. Write Through

None

Write DB + Cache together

Fresh cache, slower writes.

C. Write Back

None

Write cache first

Later flush to DB

Fast writes.

D. Refresh Ahead

Update hot keys before expiry.

11. Cache Expiry (TTL)

TTL = Time To Live

None

User profile cached for 10 min

After expiry:

None

Removed automatically

12. What to Cache

Good candidates:

None

Product catalog

User profiles

Popular posts

Session data

Search results

Configuration

Leaderboard

13. What Not to Cache

Avoid:

None

Highly dynamic balances

Real-time stock count

Sensitive temporary tokens

Critical transactional data

14. Risks / Problems

A. Stale Data

Cache may return old data.

B. Cache Stampede

Many requests after expiry hit DB together.

C. Cache Penetration

Repeated invalid keys.

D. Cache Memory Limits

RAM is expensive.

15. Important Note in HLD

Do not depend on cache for full availability.

If cache fails:

None

Requests go to DB

Need:

None

Rate limiting

Fallback logic

Autoscaling

16. Multi Data Center Concern

Cross-region cache replication is slow.

So many systems keep:

None

Local regional cache

Regional DB replicas

17. Best HLD Pattern

None

Client



CDN



API Gateway Cache



Service



Redis



SQL DB

Layered caching.

18. Interview Answer

If asked to reduce DB load:

None

Use Redis cache

Cache hot reads

Use TTL

Use cache-aside pattern

Apply rate limiting on cache failure

Strong answer.

19. Advantages

None

Fast response time

Lower DB load

Lower cost

Higher throughput

Better user experience

20. Disadvantages

None

Stale data risk

Extra complexity

Cache invalidation hard

Memory cost

Cold start misses

21. One-Line Summary

Caching stores frequently accessed data in fast memory to reduce latency, lower database load, and improve scalability, but requires careful handling of expiry and consistency.